

INTERDISZIPLINÄRES SEMESTERPROJEKT

Smarte Mülltonne 2.0

Hardware-Weiterentwicklung, Fahrverhalten und Flottenmanagement

Dokumentation des Projektstands, der Hardware-Konzeption und
der digitalen Systemerweiterung
Studiengang Digitale Technologien

Projektgruppe

Jan-Lukas Bellenhaus · Theresa Pelz · Alaeddine
Baghyour · Fulya Yasarova Bulut · Jonas Wiesner

Stand der Dokumentation

3. Mai 2026

Inhaltsverzeichnis

Einordnung und Übergabe	4
Ausgangslage bei Projektübergabe	4
Zielbild	5
Vision Statement	5
Pflichtenheft-Säulen	5
Vom Einzelprototyp zum fahrfähigen System	5
Chronologische Projektstruktur	6
Sprint 0: Orientierung und Sichtung	6
Sprint 1: Anforderungen, Architektur und Planung	6
Use Cases und Priorisierung	7
Überblick	7
Priorisierte Funktionen	7
Hardware- und Mechanikentwicklung	9
Ziel der Hardware-Weiterentwicklung	9
Fahrwerk, Räder und Ketten	9
Antriebsauslegung und Schrittmotoren	10
Sensorik und Bedienung	11
Fahrlogik und Regelung	11
Energie, Sicherheit und Elektrik	11
Bewertung für den Projektverlauf	12
Digitale Erweiterung: Flottenmanagement-Web-App	13
Rolle der Web-App im Gesamtprojekt	13
Benchmarking bestehender Smart-Waste- und Fleet-Management-Systeme	13
Ziel des Benchmarkings	13
Ausgangspunkt der Betrachtung	13
Betrachtete Referenzsysteme	14
Relevante Vergleichsdomänen	14
Vergleichsachsen für die eigene Konzeption	15
Zusammenfassende Vergleichsdimensionen	16
Bedeutung für die Systemkonzeption	16
Monorepo-Struktur	17
Architekturentscheidung: feste Standorte statt GPS	17
Architekturentscheidung: Simulation vor Hardwareintegration	17
Backend	18
Technologie und Aufbau	18
Datenmodell	18
API-Funktionen	19
Routenplanung	19

Agentische Steuerung	19
Frontend	21
Technologie und Oberfläche	21
Dashboard-Aufbau	21
Live-Datenfluss	21
Karten- und Routenvisualisierung	22
Chat-Interface	22
Simulation und Demo-Betrieb	23
Simulationsmodule	23
Zeitraffersteuerung	23
Schnittstelle zur Hardware	24
API-Vertrag	24
Command-Queue	24
Integrationsstrategie	24
Projektorganisation	25
Teamrollen	25

Einordnung und Übergabe

Ausgangslage bei Projektübergabe

Das Projekt „Smarte Mülltonne“ wurde aus einer Vorarbeit des Wintersemesters 2025/26 übernommen. Diese Vorarbeit bildet den technischen Startpunkt, wird in dieser Dokumentation aber nur als Ausgangslage beschrieben. Der Schwerpunkt dieser Dokumentation liegt auf der Weiterentwicklung im aktuellen Semester: bessere Fahreigenschaften, stabilere Mechanik, überarbeitete Sensorik, Energie- und Sicherheitsfunktionen sowie eine ergänzende Flottenmanagement-Web-App.

Bei der Übergabe lag bereits ein funktionsorientierter Prototyp mit Hardware- und Softwareanteilen vor. Die vorhandene Software enthielt unter anderem MicroPython-Code für einen Raspberry Pi Pico beziehungsweise Pico W, Module für DC-Motoren, Linienverfolgung, Ultraschallmessung, Buttons, Buzzer, LEDs, Netzwerkkommunikation und eine einfache serverseitige Missionslogik. Zusätzlich waren Schaltpläne, 3D-Druckteile, Rechnungen, Fotos, Videos und eine ältere Dokumentation vorhanden.

Kurzfassung des Ist-Zustands: Die vorherige Gruppe hatte eine einzelne smarte Mülltonne als Prototyp vorbereitet. Die Tonne konnte grundsätzlich fahren und war mit Sensorik, Motorsteuerung und lokalen Zuständen gedacht; softwareseitig existierten Pico-Client-Code und ein einfacher Serveransatz. Offen waren vor allem robuste Fahreigenschaften, zuverlässige Mechanik, verbesserter Antrieb, saubere Integration neuer Hardwarekomponenten, Energie- und Sicherheitskonzept sowie ein modernes Managementsystem für mehrere Tonnen.

Zielbild

Vision Statement

Die Projektvision beschreibt die Transformation einer passiven Mülltonne zu einem autonomen, energieautarken IoT-System. Ziel ist ein intelligentes Entsorgungsnetzwerk, das Komfort für Nutzer erhöht, Ressourcen schont und kommunale Abläufe durch datenbasierte Planung effizienter macht.

Vision: Die smarte Mülltonne soll nicht nur eine einzelne fahrende Tonne sein, sondern Teil eines Systems aus Sensorik, autonomer Bewegung, Energieüberwachung, Sicherheitslogik und zentraler Flottensteuerung.

Pflichtenheft-Säulen

Nr.	Säule	Bedeutung im Projekt
1	Intelligente Flottensteuerung	Verwaltung mehrerer Tonnen, Planung von Abholrouten, zentrale Statusübersicht und bidirektionale Kommunikation.
2	Autarkie und Energie	Ladestation beziehungsweise Docking-Konzept und Monitoring von Akku- und Ladezuständen.
3	Mobilität und Interaktion	Verbesserte Fahrdynamik, Liniennavigation, Hinderungsreaktion, Bedienung über Touchpanel und klare Nutzerfeedbacks.

Tabelle 1: Pflichtenheft-Säulen der Smarten Mülltonne 2.0

Vom Einzelprototyp zum fahrfähigen System

Der wichtigste konzeptionelle Schritt war nicht nur der Wechsel von einer isolierten Tonne zu einem Flottenmanagement, sondern zuerst die Weiterentwicklung des physischen Prototyps zu einem zuverlässig fahrenden System. Die Mülltonne muss mechanisch stabil, kontrollierbar und wiederholbar fahren, bevor die digitale Steuerung ihren Nutzen vollständig zeigen kann. Die Web-App erweitert diese Hardwarebasis um Monitoring, Planung und Demonstration, ersetzt sie aber nicht.

Chronologische Projektstruktur

Sprint 0: Orientierung und Sichtung

Sprint 0 war zunächst als Setup- und Alignment-Phase vom 14. April 2026 bis 19. April 2026 geplant. In dieser Phase standen die Sichtung der Vorarbeiten, die Rollenklärung, die Projektvision, die Tool-Auswahl und die Definition erster Use Cases im Vordergrund.

Zeitraum	Schwerpunkt	Ergebnis
14.04.– 19.04.2026	Setup & Alignment	Projektvision konkretisieren, Rollen und Schwerpunkte klären, Projektplan anlegen.
14.04.– 19.04.2026	Research	Vorarbeiten aus dem Wintersemester 2025/26 sichten und Übergabe bewerten.
15.04.– 19.04.2026	Use Cases	Grundfunktionen wie Navigation, Füllstand, Status, Serververwaltung, Sicherheit und Aufladen sammeln.

Tabelle 2: Sprint 0 nach Notion-Zeitplanung

Sprint 1: Anforderungen, Architektur und Planung

Sprint 1 war vom 20. April 2026 bis 10. Mai 2026 geplant.

Der Sprint hatte vier parallele Arbeitsrichtungen:

- Hardware und Mechanik: Antrieb, neue Räder, Kettenführung, Sensorik, Fahrwerk, Elektrik, Sicherheitskomponenten.
- Software und System: Fahrlogik, sichere Zustände, Motorsteuerung, Datenübergabe an App.
- App und UX/UI: Flottenmanagement-App, Layout, Karte, Visualisierung und Interaktionskonzept.
- Konzept und Produktstrategie: Vision, Use Cases, Storyline und Priorisierung.

Use Cases und Priorisierung

Überblick

Die Use-Case-Matrix ordnet die Projektidee in sechs Hauptkategorien:

ID	Kategorie	Ziel
UC1	Autonome Navigation	Die Mülltonne bewegt sich entlang einer definierten Linie, erkennt Hindernisse und erreicht die Bereitstellungsposition.
UC2	Füllstandsüberwachung	Der Füllstand wird gemessen, in Prozent berechnet und an Anzeige beziehungsweise Server übergeben.
UC3	Statusanzeige	Betriebszustände werden lokal über Touchpanel, Buzzer oder LED und digital in der App sichtbar.
UC4	Zentrale Serververwaltung	Daten werden gespeichert, analysiert und für Routenplanung, Steuerung und Monitoring verwendet.
UC5	Diebstahl/Fremdnutzung	Unberechtigtes Öffnen oder Manipulationen werden erkannt und als Sicherheitsereignis gemeldet.
UC6	Aufladen	Die Tonne soll in eine Ladestation fahren und ohne manuellen Eingriff geladen werden.

Tabelle 3: Use-Case-Kategorien

Priorisierte Funktionen

Aus Sicht der Umsetzung wurden besonders die Funktionen priorisiert, die den Wechsel zum Systemprototyp ermöglichen: autonome Linienfahrt, Hindernisreaktion, Füllstandsmessung, Statusübermittlung, Serverdatenverarbeitung, bidirektionale Kommunikation, Routenplanung nach Füllstand und die Simulation von Abholprozessen.

Use Case	Funktion	Priorität	Kerntechnologie
1.0	Autonome Linienfahrt	1	IR-Liniensensoren, Motorsteuerung
1.2	Dynamische Hindernisum-fahrung	1	Ultraschallsensoren seitlich
2.0	Füllstand erkennen und aus-geben	1	Ultraschallsensorik
3.4	Zustand an Flottenmanage-ment-App übermitteln	1	API-Integration
5.0	Server speichert und analy-siert Daten	1	Backend, Datenbank
5.2	Befehle und Updates	1	HTTP oder MQTT
5.4	Optimierte Routenplanung	1	Algorithmus beziehungsweise LLM-Agent
6.0	Docking-Ladestation	2	Ladekontakte, präzise Positionierung

Tabelle 4: Auszug aus der priorisierten Use-Case-Matrix

Hardware- und Mechanikentwicklung

Ziel der Hardware-Weiterentwicklung

Die Hardware ist der Kern des Projekts. Die digitale Steuerung kann nur dann überzeugend funktionieren, wenn die Mülltonne mechanisch zuverlässig fährt, sauber auf Lenkbefehle reagiert und im Testaufbau wiederholbare Ergebnisse liefert. Deshalb liegt ein wesentlicher Teil der aktuellen Weiterentwicklung auf Antrieb, Rädern, Ketten, Gewichtsverteilung, Sensorpositionierung und sicherer Elektrik.

Aus der Übergabe ergab sich: Der vorhandene Prototyp war als Demonstrator wertvoll, seine Fahreigenschaften mussten für eine robuste Weiterentwicklung aber deutlich verbessert werden. Besonders kritisch sind Geradeauslauf, Kurvenverhalten, Traktion, Kettenführung, Spiel in der Mechanik und die Abstimmung zwischen Motoren, Sensorik und Software-Regelung.

Hardware-Schwerpunkt: Die Mülltonne soll nicht nur „irgendwie fahren“, sondern kontrolliert, reproduzierbar und sicher entlang einer Linie navigieren. Neue Motoren, gedruckte Räder, angepasste Ketten und eine überarbeitete Mechanik sind deshalb zentrale Projektbestandteile.

Fahrwerk, Räder und Ketten

Ein großer praktischer Arbeitsbereich ist die Verbesserung des Fahrwerks. Die vorhandene Kettenlösung bietet grundsätzlich gute Traktion, und wird von uns im Rahmen dieses Projektes weiterentwickelt. Bei der Analyse des IST-Zustands ist uns am Fahrwerk ein deutliches Spiel aufgefallen, welches die Fahreigenschaften negativ beeinflusst. Auch der verwendete DC-Motor ist für die Anforderungen an präzise Linienführung und kontrollierte Kurvenfahrt nicht optimal.

Für die Weiterentwicklung werden deshalb neue Räder beziehungsweise Antriebselemente konstruiert und gedruckt. Ziel ist, dadurch mehr Platz für die Integration eines besseren Schrittmotors zu schaffen. Dafür werden die vorhandenen Ketten durch einen weiteren, baugleichen Satz erweitert. Dadurch wird nicht nur eine Integration eines stärkeren Motors ermöglicht, sondern auch die Traktion verbessert, da die Kraft auf mehr Kontaktpunkte verteilt wird. Gleichzeitig soll die Kettenführung überarbeitet werden, um Abspringen oder Verkanten zu reduzieren, besonders bei Kurven und Drehungen. Das Fahrwerk wird insgesamt überarbeitet, um die Einbaupositionen für Motoren, Sensorik, Akku, Touchpanel und Solarfläche sauber festzulegen. Schließlich wird die Gewichtsverteilung geprüft und angepasst, um Kippen, Schleifen und einseitige Belastung zu vermeiden.

Baustelle	Ziel der Überarbeitung
Neue Räder drucken	Bessere Passung zum Fahrwerk, stabilere Kraftübertragung und reproduzierbarere Bewegung.
Ketten erweitern/anpassen	Mehr Traktion, weniger Durchrutschen und stabileres Fahrverhalten auf dem Testuntergrund.
Kettenführung verbessern	Abspringen oder Verkanten reduzieren, besonders bei Kurven und Drehungen.
Fahrwerk überarbeiten	Einbaupositionen für Motoren, Sensorik, Akku, Touchpanel und Solarfläche sauber festlegen.
Gewichtsverteilung prüfen	Kippen, Schleifen und einseitige Belastung vermeiden.

Tabelle 5: Mechanische Schwerpunkte zur Verbesserung der Fahreigenschaften

Antriebsauslegung und Schrittmotoren

Die Antriebsauslegung bewertet die vorhandenen DC-Motoren und vergleicht sie mit möglichen Schrittmotoren. Hintergrund ist, dass die bisherige Fahrweise zwar demonstrierbar war, für präzise Liniennavigation, kontrollierte Kurven und wiederholbare Manöver aber mehr Reserven und eine feinere Ansteuerbarkeit benötigt werden.

Für ein Kettenrad mit 70 mm Teilkreisdurchmesser und eine angenommene Zugkraft von rund 10 N ergibt sich ein Drehmoment am Antriebsrad von etwa 0,35 Nm. Auf zwei Motoren verteilt entspricht dies etwa 0,175 Nm pro Motor.

Größe	Wert	Bedeutung
Zugkraft mit Sicherheitsfaktor	ca. 10 N	Grundlage für Drehmomentabschätzung.
Drehmoment am Antriebsrad	ca. 0,35 Nm	Benötigtes Moment am Rad bei 35 mm Radius.
Drehmoment pro Motor	ca. 0,175 Nm	Verteilung auf zwei Antriebe.
Drehzahl bei 0,3 m/s	ca. 82 U/min	Langsame, kontrollierte Bewegung.
Drehzahl bei 0,5 m/s	ca. 136 U/min	Schnellere Fahrt im Testbetrieb.

Tabelle 6: Auszug aus der Motorauslegung

Als mögliche Schrittmotor-Optionen wurden ein NEMA17 42BYGHM809 und ein stärkerer NEMA17-05GM betrachtet. Die Berechnung zeigt, dass der stärkere Motor mit nutzbarem Drehmoment von ungefähr 0,504 bis 0,840 Nm eine deutlich höhere Reserve bietet. Der Schrittmotor ist für das Projekt besonders interessant, weil er sich exakter ansteuern lässt und dadurch bessere Voraussetzungen für kontrollierte Drehungen, gleichmäßige Geschwindigkeit und reproduzierbare Fahrmanöver schafft.

Komponente	Status/Plan	Relevanz
NEMA17-Schrittmotor	geplant/geprüft	Mehr Drehmomentreserve und präzisere Bewegungssteuerung als reine DC-Ansteuerung.
Schrittmotor-Treiber	recherchiert	Notwendig für saubere Ansteuerung, Strombegrenzung und Microstepping.
Gedruckte Räder	in Planung/Umsetzung	Mechanische Kopplung zwischen Motor, Achse und Kette verbessern.
Kettenchassis	vorhanden, wird überarbeitet	Grundlage der Mobilität, aber abhängig von sauberer Führung und Spannung.

Tabelle 7: Antriebskomponenten und Rolle im Projekt

Sensorik und Bedienung

Für die Weiterentwicklung wurden zusätzliche oder überarbeitete Sensorik- und Bedienkomponenten geplant. Dazu gehören Magnetschalter für Deckelöffnung beziehungsweise Fremdnutzung, ein Touchdisplay als lokale Schnittstelle, Not-Aus, Buzzer, Akkuüberwachung, ein Spannungswandler und eine Solar- beziehungsweise Ladeinfrastruktur.

Die Notion-Materialliste führt außerdem vorhandene Grundkomponenten wie Kettenroboterchassis, Mülltonnenkorpus, Motoren, Motortreiber und Bleiakku. Als Randbedingung wurde unter anderem ein Gesamtbudget und die verfügbare GPIO-Anzahl betrachtet; in der Materialliste ist festgehalten, dass 34 GPIO-Ports benötigt werden.

Fahrlogik und Regelung

Die mechanische Verbesserung ist eng mit der Fahrlogik gekoppelt. Die vorhandene Software arbeitet mit Linienverfolgung, PD-Regelung, Hinderniserkennung, Pausen- und Startzuständen sowie Netzwerkkommunikation. Für die Weiterentwicklung bedeutet das: Neue Motoren und Räder können nicht isoliert betrachtet werden. Nach mechanischen Änderungen müssen Parameter wie Grundgeschwindigkeit, Regelverstärkung, Trim der beiden Antriebsseiten und Verhalten bei Linienverlust erneut getestet werden.

Die wichtigsten fahrtechnischen Ziele sind:

- stabiler Geradeauslauf ohne starkes Pendeln
- weichere Kurvenfahrt entlang der Linie
- kontrollierte 180-Grad-Drehung an Ziel- oder Wendepunkten
- sicherer Stopp bei Hindernissen
- Wiederfinden der Linie nach Störung oder Ausweichbewegung
- klare sichere Zustände bei Not-Aus, Sensorfehler oder Kommunikationsverlust

Energie, Sicherheit und Elektrik

Neben dem Fahrwerk werden auch Energieversorgung und Sicherheit weitergedacht. Die geplante Dockingstation soll langfristig ermöglichen, dass die Tonne ohne manuelles Laden einsatzfähig bleibt. Gleichzeitig sind Magnetschalter, Not-Aus, Buzzer/LED und Statusanzeigen relevant, weil die Tonne im öffentlichen oder halböffentlichen Raum zuverlässig und nachvollziehbar reagieren muss.

Bereich	Geplante Funktion
Solar und Akku	Betriebsdauer erhöhen, Ladestand messen und in App beziehungsweise Display anzeigen.
Dockingstation	Automatisches Laden über Kontaktflächen als Zukunftsschritt.
Magnetschalter	Deckelöffnung erkennen und Fremdnutzung melden.
Not-Aus	Physischer, hardwareunabhängiger Sicherheitsstopp.
Touchpanel	Lokale Bedienung und Statusanzeige direkt an der Tonne.
Buzzer/LED	Akustisches und visuelles Feedback bei Fehlern, Hindernissen oder Statuswechseln.

Tabelle 8: Hardwarefunktionen neben dem Antrieb

Bewertung für den Projektverlauf

Die Hardwareentwicklung ist der entscheidende Erfolgsfaktor des Projekts. Routenplanung, Sicherheitslogik und Energieanzeige entfalten ihren Wert erst, wenn die Mülltonne als physischer Prototyp zuverlässig fährt und sinnvoll Daten liefern kann. Gleichzeitig ist die Hardwareintegration risikoreich, da Motorsteuerung, Räder, Ketten, Sensorik, Energieversorgung und Kommunikation zeitlich eng gekoppelt sind.

Technische Risiken: Die wichtigsten Risiken liegen in der sauberen Linienverfolgung, der Kettenführung, der mechanischen Passung gedruckter Räder, der stabilen Stromversorgung, der Anzahl benötigter GPIOs, der zuverlässigen WLAN-Kommunikation und der Synchronisation zwischen lokaler Fahrlogik und zentralen Serverbefehlen.

Digitale Erweiterung: Flottenmanagement-Web-App

Rolle der Web-App im Gesamtprojekt

Die Web-App wurde als ergänzender Prototyp entwickelt. Während Hardware, Mechanik und Fahreigenschaften die Grundlage bilden, zeigt die App, wie die Tonne später in ein größeres System eingebunden werden kann: mit mehreren Tonnen, Füllstand, Akku, Ladeleistung, Sicherheitsstatus, Position, Routenplanung und Live-Visualisierung.

Die App dient damit als Demonstrator, Testumgebung und spätere Integrationsschicht zur Hardware. Sie ist gewissermaßen das sichtbare Management-Layer über dem eigentlichen mechatronischen System.

Benchmarking bestehender Smart-Waste- und Fleet-Management-Systeme

Ziel des Benchmarkings

Im Rahmen der Konzeption der Flottenmanagement-Web-App wurde ein Benchmarking bestehender Systeme aus den Bereichen Smart Waste Management und Robot Fleet Management durchgeführt. Ziel war es, relevante funktionale und gestalterische Muster existierender Plattformen zu identifizieren, ihre Übertragbarkeit auf das eigene Projekt zu prüfen und daraus Anforderungen, Strukturprinzipien sowie Entwicklungsperspektiven für die eigene Web-Anwendung abzuleiten.

Die Web-App ist im Projekt als digitaler Management-Layer vorgesehen. Sie soll mehrere smarte Mülltonnen visualisieren, überwachen und perspektivisch operative Entscheidungen unterstützen. Damit ergänzt sie den mechatronischen Kern des Projekts, also die fahrende, sensorisch erfasste und energieüberwachte Mülltonne.

Ausgangspunkt der Betrachtung

Die digitale Erweiterung wurde nicht als isolierte Einzelansicht entworfen, sondern als Flottenmanagementsystem im Kleinen. Sie soll den Zustand mehrerer Tonnen in Echtzeit sichtbar machen, Routen visualisieren, Energie- und Sicherheitszustände abbilden und eine Schnittstelle zwischen simulierten beziehungsweise später real integrierten Hardwarekomponenten und einem zentralen Managementsystem herstellen.

Für das Benchmarking wurden deshalb nicht nur klassische Smart-Waste-Plattformen betrachtet, sondern auch Systeme aus dem Bereich Robot Fleet Management. Diese Systeme

adressieren ähnliche Herausforderungen: zentrale Leitstellenlogik, Visualisierung verteilter Einheiten, Health-Monitoring, Routen- und Aufgabenkoordination sowie Verarbeitung von Live-Daten in webbasierten Dashboards.

Betrachtete Referenzsysteme

System	Domäne	Relevante Funktionen	Relevanz für unser Projekt
BrighterBins	Smart Waste Management	Echtzeit-Dashboard, Füllstandsanzeige, Alarme, Routenoptimierung.	Referenz für die Verbindung aus Behältermonitoring und operativer Tourenplanung.
ThingsBoard / IoT Hub	IoT / Smart Waste	Live-Dashboards, Rule Engine, Multi-Tenant-Strukturen, Sensordatenintegration.	Relevant für die zentrale Visualisierung und Verarbeitung von Zustandsdaten mehrerer Einheiten.
Renderbit	Smart City / Waste Tracking	Live-Karte, Fahrzeug- und Behältertracking, farbliche Statuscodierung, Historical Playback.	Besonders relevant für kartenzentrierte Leitstellenlogik und visuelle Flottendarstellung.
MiR Fleet	Robot Fleet Management	Zentrale Flottensteuerung, Task-Zuweisung, Statusmonitoring.	Referenz für eine Leitstellenlogik für mehrere autonome Einheiten.
InOrbit	Robot Operations	Echtzeit-Flottenübersicht, Incident-Logik, Health-Monitoring, Verlaufsdaten.	Relevant für die Kombination aus Monitoring, Ereignismanagement und operativer Übersicht.
KUKA.AMR Fleet Manager	AMR-Flottenmanagement	Routenkoordination, Prozessstatus, Flottensteuerung, Integrationsschnittstellen.	Zeigt, wie komplexere Flottenmanagementsysteme als operative Steuerungsebene aufgebaut sind.
Boston Dynamics Orbit	Robot Fleet / Site Operations	Standortübergreifende Dashboards, Flottengesundheit, Performance-Monitoring.	Referenz für die Aggregation von Zustandsdaten in einer zentralen Oberfläche.

Tabelle 9: Betrachtete Referenzsysteme im Benchmarking

Aus dem Vergleich der betrachteten Systeme wurde deutlich, dass sich unabhängig von der jeweiligen Domäne mehrere wiederkehrende Funktionsblöcke identifizieren lassen. Dazu gehören insbesondere eine zentrale Flottenübersicht, die Verdichtung von Telemetrie- und Statusdaten, Funktionen zur Einsatz- und Routenkoordination sowie Mechanismen zur Ereignis- und Sicherheitsbehandlung.

Relevante Vergleichsdomänen

Smart-Waste-Management-Systeme sind besonders nah an der eigenen Projektidee, da sie Behälterzustände, Füllstandssensorik, Sammelprozesse und kommunale Entsorgungslogik digital unterstützen. Die betrachteten Systeme zeigen wiederkehrend dieselben Kernmuster: eine zentrale Übersicht über Behälter und Fahrzeuge, Darstellung von Füllstän-

den und Zuständen in Echtzeit, Alarmer bei kritischen Ereignissen sowie Unterstützung bei Touren- und Sammelprozessen.

Robot-Fleet-Management-Systeme sind nicht direkt dem Entsorgungsbereich zuzuordnen, zeigen jedoch sehr deutlich, wie verteilte autonome Einheiten zentral überwacht, koordiniert und in operative Prozesse eingebunden werden können. Für das eigene Projekt waren sie relevant, weil sie starke Vorbilder für Dashboard-Architektur, Flottenübersicht, Zustandsaggregation, Aufgabensteuerung und Leitstellenlogik liefern.

Vergleichsachsen für die eigene Konzeption

Ein zentrales Muster der Benchmark-Systeme ist die Kombination aus Kartenansicht und Flottenliste. Position und Status einzelner Einheiten werden meist über Marker auf einer Karte beziehungsweise einer räumlichen Oberfläche sowie ergänzend in Listen, Statuskarten oder Panels dargestellt. Diese Beobachtung war direkt anschlussfähig an die eigene Web-App, in der FleetPanel, MapView, Route, Depot und Truck bewusst nebeneinander gedacht wurden.

Ein weiteres Ergebnis war die hohe Bedeutung verdichteter Betriebs- und Zustandsdaten. In Smart-Waste-Systemen stehen Füllstände, Batterie- oder Energiezustände, Konnektivität und Sammelstatus im Vordergrund. In Robot-Fleet-Systemen treten Health-Indikatoren wie Batterie, Verfügbarkeit, Incident-Status oder Netzwerkzustände an ihre Stelle. Für das eigene Projekt bestätigte dies die Konzentration auf Füllstand, Akku, Solarleistung, Sicherheitszustand und Sperrstatus als zentrale Telemetriedaten.

Auch Routen- und Einsatzplanung erscheinen in professionellen Systemen als Kernfunktion. Für das eigene Projekt war dabei weniger entscheidend, welcher konkrete Optimierungsalgorithmus in der frühen Prototypenphase genutzt wird. Wichtiger war die Erkenntnis, dass Routing grundsätzlich datengetrieben, dynamisch und erweiterbar gedacht werden sollte. Die aktuelle Tourenlogik ist daher als funktionaler Prototyp zu verstehen. Perspektivisch sind stärkere Optimierungsverfahren wie 2-Opt, verbesserte Tourenheuristiken oder Python-basierte Routing-Frameworks anschlussfähig und wurden auch im Austausch mit dem betreuenden Professor als sinnvoller Entwicklungspfad diskutiert.

Die Analyse bestehender Systeme zeigte außerdem, dass professionelles Flottenmanagement fast immer Ereignis- und Alarmmanagement umfasst. In Smart-Waste-Lösungen betrifft dies beispielsweise Überfüllung, Manipulation, Feuer oder Fremdnutzung. In Robotiksystemen geht es eher um Incidents, technische Ausfälle, Kommunikationsprobleme oder Zustandsverletzungen. Diese Einordnung stärkte die Entscheidung, Sicherheitsereignisse, Sperrlogik, Alert-Banner und Quittierungsmechanismen in die eigene Lösung aufzunehmen.

Besonders relevant war zudem die Rolle von Simulation und Demo-Betrieb. Viele professionelle Systeme verfügen über Verlaufsansichten, historische Auswertungen oder Testumgebungen. Für das eigene Projekt wurde früh entschieden, die digitale Schicht auch unabhängig von vollständig integrierter Hardware entwickelbar zu machen. Simulation

wurde daher nicht als Notlösung verstanden, sondern als sinnvoller Entwicklungspfad für ein cyber-physisches System.

Zusammenfassende Vergleichsdimensionen

Vergleichsdimension	Beobachtung in Benchmark-Systemen	Relevanz für das Projekt
Flottenübersicht	Zentrale Kombination aus Karte, Flottenliste und Statusanzeige.	Bestätigung der eigenen Dashboard-Struktur mit FleetPanel und MapView.
Telemetrie	Echtzeitdarstellung von Füllstand, Batterie, Status und Konnektivität.	Bestätigung der Fokussierung auf Füllstand, Akku, Solar und Sicherheitsstatus.
Routenplanung	Routing und Einsatzplanung sind Kernbestandteil professioneller Systeme.	Routing wurde früh als ausbaufähige Kernfunktion integriert.
Sicherheitslogik	Alerts, Incidents und Ereignismanagement gehören zum Standard.	Bestätigung von Sicherheitsereignissen, Sperren und Quittierungen.
Simulation / Historie	Historisierung, Testumgebungen und Playback-Funktionen sind verbreitet.	Bestätigung der Strategie „Simulation vor Hardwareintegration“.
Leitstellenlogik	Zentrale operative Oberflächen verdichten Informationen und ermöglichen Eingriffe.	Einordnung der Web-App als Management-Layer und nicht nur als Visualisierung.

Tabelle 10: Vergleichsdimensionen und Ableitungen für die eigene Konzeption

Bedeutung für die Systemkonzeption

Aus Sicht der Projektgruppe hatte das Benchmarking vor allem drei Funktionen. Erstens half es dabei, die Web-App nicht nur als projektspezifische Einzelentwicklung zu betrachten, sondern als Teil eines größeren Musters digitaler Flottenmanagementsysteme. Zweitens zeigte es, dass Flottenübersicht, Statusdarstellung, Routenlogik, Sicherheitsereignisse und Simulation keine isolierten Ideen sind, sondern sich eng an wiederkehrenden Strukturen realer Systeme orientieren. Drittens machte das Benchmarking sichtbar, an welchen Stellen der Prototyp langfristig weiterentwickelt werden kann: stärkere Tourenoptimierung, Rollenmodelle, historische Auswertungen, Replay-Funktionen und vertiefte agentische Unterstützung.

Das Benchmarking war damit kein Mittel zur nachträglichen Bewertung der eigenen Lösung, sondern ein konzeptionelles Werkzeug zur Orientierung und Einordnung. Es positioniert die Web-App als nachvollziehbaren Prototyp innerhalb eines professionellen Systemkontexts und zeigt zugleich Entwicklungsperspektiven für den weiteren Ausbau.

Monorepo-Struktur

Ordner	Rolle
backend/	FastAPI-Backend mit SQLite-Datenbank, REST-Endpunkten, WebSocket, Routenplanung und Agent.
frontend/	Next.js-15-Dashboard mit Karte, Fleet Panel, Chat, Energie- und Sicherheitsansicht.
simulator/	Python-Simulatoren für Füllstand, Energie, Sicherheitsereignisse und Abholfahrzeug.
docs/	Architektur- und API-Vertrag zwischen Backend und späterer Hardware.
docker-compose.yml	Lokale Orchestrierung von Backend und Frontend.

Tabelle 11: Struktur des neuen smart-bin-Prototyps

Architekturentscheidung: feste Standorte statt GPS

Eine zentrale Entscheidung war, auf GPS-Ortung zu verzichten. Mülltonnen stehen in der Regel an bekannten Adressen. Deshalb werden Koordinaten einmalig in der Datenbank hinterlegt. Die Hardware muss später nur Statusdaten wie Füllstand, Akku, Sicherheitsereignisse und gegebenenfalls Fahrzustand übertragen.

Entscheidung: Die Web-App modelliert Tonnen über feste Adressen und Koordinaten. Dadurch bleibt die Hardware einfacher, und die Flottenlogik kann trotzdem echte Karten- und Routenfunktionen nutzen.

Architekturentscheidung: Simulation vor Hardwareintegration

Da die echte Hardware nicht durchgehend verfügbar beziehungsweise noch nicht vollständig integriert ist, simuliert smart-bin mehrere Systemteile. Die Simulation ist kein Ersatz für echte Tests, aber sie erlaubt eine frühzeitige Entwicklung der App-Logik: Live-Daten, Routen, Truck-Bewegung, Füllstandsdynamik, Solarwerte und Sicherheitsalerts können ohne angeschlossene Tonne geprüft werden.

Backend

Technologie und Aufbau

Das Backend basiert auf FastAPI, SQLAlchemy, SQLite und Pydantic Settings. Beim Start initialisiert es die Datenbank und legt Beispieldaten für Tonnen im Raum Soest an. Die API ist in Router für Tonnen, Routen, Sicherheit, Energie, Fahrzeug, Commands, Agent, Simulation und WebSocket aufgeteilt.

Modul	Aufgabe
main.py	Initialisiert FastAPI, CORS, Datenbank-Lifespan und Router.
database.py	SQLAlchemy-Engine, Session-Verwaltung und Seed-Daten für Soester Tonnen.
models/	Datenmodelle für Tonnen, Routen, Commands und Sicherheitsereignisse.
routers/	REST-, SSE- und WebSocket-Endpunkte.
services/routing.py	OSRM-Anbindung für echte Straßenrouten mit Fallback auf Luftlinie.
agent/	Tool-calling-Agent für natürliche Sprache und operative Aktionen.

Tabelle 12: Backend-Bausteine

Datenmodell

Das Datenmodell ist bewusst klein gehalten, deckt aber die Kernobjekte des Systems ab.

Tabelle	Inhalt
bins	Name, Adresse, Koordinaten, Füllstand, Akku, Solarleistung, Ladezustand, Status, Sperrstatus und letztes Update.
routes	Route mit geordneten Tonnen-IDs, Distanz, Dauer, GeoJSON-Geometrie, Abschlussstatus und optionaler Agentenbegründung.
security_events	Manipulations- oder Fremdnutzungsereignisse mit Tonne, Typ, Zeitstempel und Quittierungsstatus.
commands	Warteschlange für Kommandos an einzelne Tonnen, inklusive Parametern und ACK-Zeitpunkt.

Tabelle 13: Persistente Kernobjekte

API-Funktionen

Die REST-API ist so aufgebaut, dass sie sowohl vom Frontend als auch später von einem Raspberry Pi oder Pico-Gateway genutzt werden kann.

Endpunktgruppe	Funktion
/bins	Alle Tonnen abrufen, einzelne Tonne abrufen, partielle Statusupdates schreiben.
/routes	Route planen, letzte Route abrufen und Route als abgeschlossen markieren.
/security	Sicherheitsereignisse abrufen oder erzeugen, Tonne sperren, entsperren und Alerts quittieren.
/energy	Akku-, Solar- und Ladezustände abrufen, Docking-Status setzen.
/truck	Simulierten Truck-Status lesen, Position schreiben und Start/Pause/Stop setzen.
/bins/{id}/command	Kommandos an eine Tonne einreihen, pending Commands pollen und ACKs zurückmelden.
/agent/chat	Server-Sent-Events-Streaming für den Chat-Agenten.
/ws/live	Live-Payload aus Tonnen, Alerts und Truck im Zwei-Sekunden-Takt.

Tabelle 14: API-Überblick

Routenplanung

Die Routenplanung filtert zunächst gesperrte Tonnen und Tonnen unterhalb eines Füllstand-Schwellwerts von 60 Prozent heraus. Danach wird ab dem Depot eine Nearest-Neighbor-Reihenfolge gebildet. Für die resultierende Rundfahrt wird über OSRM eine echte Straßenroute mit Distanz, Dauer und GeoJSON-Geometrie angefragt. Falls OSRM nicht erreichbar ist, erzeugt das Backend eine gerade Fallback-Route.

Diese Lösung ist für den Prototyp pragmatisch: Sie ist nachvollziehbar, schnell implementierbar und zeigt bereits den Nutzen von datenbasierter Abholplanung. Für eine spätere Version könnte der Algorithmus durch eine bessere Optimierung oder externe Tourenplanung ersetzt werden.

Agentische Steuerung

Der Agent im Backend nutzt LangChain mit Groq beziehungsweise Llama 3.3 70B und kann über Tools echte Backend-Aktionen auslösen. Er kann Tonnenstatus abrufen, Routen planen, den Truck starten, Tonnen sperren oder entsperren, Sicherheitsereignisse prüfen und Energiedaten auswerten.

Die Systemregeln priorisieren Tonnen mit hohem Füllstand, behandeln Akku als Monitoring-Wert und reagieren bei Manipulationsereignissen mit Sperrlogik. Der Agent ist damit nicht nur ein Chatbot, sondern eine natürliche Bedienoberfläche für operative Flottenaktionen.

Frontend

Technologie und Oberfläche

Das Frontend basiert auf Next.js 15, React 19, TypeScript, Tailwind CSS 4, React Leaflet, OpenStreetMap und Lucide Icons. Die Startseite leitet direkt auf /dashboard weiter. Das Dashboard ist als Arbeitsoberfläche aufgebaut und besteht aus Live-Status, Kartenansicht und Seitenpanel.

Dashboard-Aufbau

Bereich	Funktion
Header	Titel, Live-Status, Simulationsgeschwindigkeit und Button zur Routenplanung.
AlertBanner	Roter Warnbereich bei offenen Sicherheitsereignissen.
FleetPanel	Liste aller Tonnen sortiert nach Füllstand mit Akku-, Lade- und Sperrstatus.
MapView	OpenStreetMap-Karte mit Tonnenmarkern, Depot, Route und animiertem Truck.
Chat	Streaming-Chat mit Tool-Call-Anzeige und Quick Prompts.
Energie	Gesamtsolarleistung, durchschnittlicher Akku und Detailwerte je Tonne.
Sicherheit	Offene Ereignisse, Sperren und Quittieren.

Tabelle 15: Frontend-Komponenten

Live-Datenfluss

Die Web-App erhält Live-Daten über WS /ws/live. Der Hook useLiveData() stellt automatisch eine WebSocket-Verbindung her, verarbeitet JSON-Payloads und reconnectet nach Verbindungsabbrüchen. Dadurch bleiben Karte, Fleet Panel, Energieansicht und Sicherheitsansicht konsistent.

Live-Payload: Das Backend sendet alle zwei Sekunden Tonnenliste, offene Alerts und optional die Truck-Position. Das Frontend rendert daraus Marker, Warnungen, Statuskarten und Bewegungsanimationen.

Karten- und Routenvisualisierung

Die Karte nutzt OpenStreetMap-Tiles und React Leaflet. Tonnen werden als farbcodierte Marker dargestellt: Grün für niedrigen, Gelb für mittleren und Rot für hohen Füllstand. Kritisch volle Tonnen erhalten eine visuelle Pulsierung; gesperrte Tonnen werden grau dargestellt. Eine geplante Route wird als GeoJSON-Linie gerendert, wenn OSRM-Geometrie vorhanden ist. Der simulierte Truck wird als eigener Marker animiert und zwischen WebSocket-Updates geglättet.

Chat-Interface

Das Chat-Interface ist an den Agent-Endpunkt angebunden und verarbeitet Server-Sent Events. Text wird tokenweise gestreamt, Tool-Aufrufe werden als eigene Karten angezeigt. Dadurch ist sichtbar, wann der Agent tatsächlich eine Route plant, den Truck startet oder Sicherheitsaktionen ausführt.

Simulation und Demo-Betrieb

Simulationsmodule

Die Simulatoren ermöglichen eine vollständige Demo ohne angeschlossene Hardware.

Script	Simuliertes Verhalten
<code>mock_bins.py</code>	Füllstände steigen langsam an und skalieren mit der Demo-Geschwindigkeit.
<code>mock_energy.py</code>	Solarleistung folgt einer Tageszeit-Sinuskurve, Akkus laden oder entladen sich.
<code>mock_security.py</code>	Erzeugt zufällige Manipulations- oder Fremdnutzungsereignisse.
<code>mock_truck.py</code>	Fährt entlang der letzten OSRM-Route, leert Tonnen beim Erreichen und markiert die Route als abgeschlossen.

Tabelle 16: Simulationsbausteine

Zeitraffersteuerung

Über `/sim/speed` kann die Demo-Geschwindigkeit zwischen 0,5-fach und 20-fach gesetzt werden. Das Frontend bietet dafür Buttons mit 1x, 5x, 10x und 20x. Die Simulatoren pollen den Wert und passen Tick-Rate beziehungsweise Bewegungsfortschritt an. Damit lassen sich Routen, Füllstandsdynamik und Abholprozesse in kurzer Zeit zeigen.

Schnittstelle zur Hardware

API-Vertrag

Der API-Vertrag in `docs/api_contract.md` definiert, wie ein Raspberry Pi oder ein Hardware-Gateway später mit dem Backend sprechen soll. Tonnen senden regelmäßig oder bei Änderungen Statusupdates an `/bins/{bin_id}/update`. Sicherheitsereignisse werden sofort an `/security/events` gesendet.

Beispielhafte Statusdaten:

```
{
  "fill_level": 87,
  "battery": 64,
  "timestamp": "2026-04-10T14:32:00Z"
}
```

Optionale Energiedaten können Solarleistung und Ladezustand ergänzen:

```
{
  "fill_level": 87,
  "battery": 64,
  "solar_output_w": 11.4,
  "is_charging": true
}
```

Command-Queue

Für Befehle an einzelne Tonnen existiert eine Command-Queue. Das Backend kann Aktionen wie `lock`, `unlock`, `empty` oder spätere Spezialbefehle einreihen. Eine Tonne beziehungsweise ein Gateway kann regelmäßig `pending-command` abfragen und anschließend per ACK bestätigen. Diese Polling-Logik ist robust genug für einen Prototyp, weil die Hardware nicht dauerhaft eine eingehende Serververbindung halten muss.

Integrationsstrategie

Die vorgesehene Integrationsstrategie ist inkrementell:

1. Hardware sendet zunächst nur Füllstand und Akku.
2. Sicherheitsereignisse werden als eigener Pfad ergänzt.
3. Commands werden über Polling und ACKs eingebunden.
4. Fahrzustände und Position beziehungsweise Zustandsübergänge werden in die Live-Daten aufgenommen.
5. Danach wird getestet, ob der Agent operative Aktionen zuverlässig auslösen darf.

Projektorganisation

Teamrollen

Lorem Ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur. Donec ut libero sed arcu vehicula ultricies a non tortor. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec a diam lectus. Sed sit amet ipsum mauris. Maecenas congue ligula ac quam viverra nec consectetur ante hendrerit. Donec et mollis dolor. Praesent et diam eget libero egestas mattis sit amet vitae augue. Nam tincidunt congue enim, ut porta lorem lacinia consectetur.

Team	Aufgaben	Personen laut Board
Hardware & Mechanik	Antrieb, Konstruktion, Sensoren, Solar, Elektronik.	Jan-Lukas, Theresa, Alaeddine
Software & System	Pico-Code, Kommunikation, Backend, Datenverarbeitung, Navigation.	Jan-Lukas, Jonas, Theresa, Alaeddine
App & UX/UI	App-Design, User Flows, Touchpanel, Feedbacksysteme, Usability.	Fulya, Jonas
Konzept & Produktstrategie	Use Cases, Vision, Flottenmanagement-Konzept, Storytelling.	Fulya, Jonas

Tabelle 17: Rollenmodell aus dem Notion-Board